

# Software Testing Report

Moderation System • Restaurant Review Platform • Version V1.0

## Chapter I: Introduction

### 1.1 Purpose

This Testing Report contains detailed information relating to the testing plans and results for the Moderation System of the Restaurant Review Platform. This document replaces the previous subsystem data and provides the basis for the testing team to ensure all moderation components fulfill the requirements specified in the Software Design Document (SDD).

### 1.2 Scope

- **Name:** Restaurant Review Platform - Moderation System.
- **Users:** Users, Vendors, and Moderators.
- **Network Environment:** Cloud-based deployment utilizing a layered architecture (API, Service, Repository, Database).

### 1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface.
- **JWT:** JSON Web Token, used for authenticating users.
- **ORM:** Object-Relational Mapper (Prisma ORM is used).
- **RBAC:** Role-Based Access Control, utilized to restrict functionality based on the user's role.

### 1.4 Overview

The Moderation System testing encompasses five core components:

1. **Restaurant Verification:** Automatic verification based on customer review metrics.
2. **Media Moderation:** Reviewing images uploaded by users/vendors (moving from private to public storage).
3. **Review Moderation:** Enforcing review policies and managing reported reviews.

4. **Report Management:** Handling policy violation reports from submission to resolution.
5. **Customer Support Management:** Processing support requests through a ticket system.

## Chapter II: Testing Plan Overview

---

### 2.1 General Description

To validate the layered architecture of the Moderation System, the system is tested across its endpoints and service layers. The technology stack under test includes Express.js, PostgreSQL, Prisma ORM, and Zod validation.

### 2.2 Test Environment

- **Backend:** TypeScript/JavaScript running on Express.js.
- **Database:** PostgreSQL accessed via Prisma ORM.
- **Validation & Auth:** Zod for input validation, JWT for token-based authentication.

### 2.3 Schedule & Test Types

1. **Module Test:** Validating individual component logic.
2. **Marginal Value Test:** Testing edge cases (e.g., exact rating thresholds).
3. **Stress Test:** Simulating high-volume reporting and media uploads.
4. **External API Test:** Validating standard RESTful interactions.
5. **Security Test:** Ensuring JWT and RBAC integrity.

## Chapter III: Module Test

### 3.1 Component Input and Expected Output

| Module                  | Input / Trigger                                                                | Expected Output                                                                                                                 | Status |
|-------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------|
| Restaurant Verification | A new review is submitted via the Review Service.                              | The system checks if criteria are met. If met, status updates to <b>Verified</b> in PostgreSQL.                                 | PASS   |
| Media Moderation        | Moderator selects to approve or remove pending media.                          | Media is moved to the public bucket (if approved) or removed (if rejected); notes are recorded.                                 | PASS   |
| Review Moderation       | Moderator reviews reported content via <code>/moderation/reviews/{id}</code> . | Review is approved or deleted; moderation decision is recorded.                                                                 | PASS   |
| Report Management       | Customer/Vendor submits a report against a restaurant, review, or media.       | A report is generated with <b>Pending</b> status. Invalid requests (e.g., reporting own review) are rejected.                   | PASS   |
| Customer Support        | Customer/Vendor creates a support ticket.                                      | Ticket is created with <b>Pending</b> status. Can be transitioned to <b>In Progress</b> , <b>Resolved</b> , and <b>Closed</b> . | PASS   |

## Chapter IV: Marginal Value (Boundary) Test

### 4.1 Verification Criteria Boundary Tests

The Restaurant Verification component utilizes strict numerical rules that must be tested at their boundaries.

- **Rule 1: At least five reviews.**

- *Input:* 4 reviews. *Result:* Remains **Unverified**.
- *Input:* 5 reviews. *Result:* Triggers verification logic.

- **Rule 2: Reviews by at least five different users.**

- *Input:* 5 reviews, but 2 are from the same user. *Result:* Remains **Unverified**.

- *Input:* 5 reviews from 5 distinct users. *Result:* Proceeds to rating check.

- **Rule 3: Average rating  $\geq 2.5$ .**

- *Input:* Average rating 2.49. *Result:* Remains **Unverified**.

- *Input:* Average rating 2.5. *Result:* Status updates to **Verified**.

## 4.2 Report Management Rule Tests

- **Rule: Vendors cannot report their own restaurants.**

- *Input:* Vendor ID matches target Restaurant Vendor ID. *Result:* Rejected (400 Bad Request).

- **Rule: Duplicate reports for the same target by the same user are not permitted.**

- *Input:* User submits duplicate **/reports** POST payload. *Result:* Rejected (409 Conflict).

# Chapter V: Stress Test

---

## 5.1 Load Testing API Endpoints

Stress testing evaluates the system's ability to handle high-volume moderation requests.

- **Target:** **/reports** (POST) and **/moderation/media** (GET).
- **Condition:** 10,000 concurrent report submissions and media queue queries.
- **Result:** The Express.js application and Prisma ORM handled connection pooling successfully. Input validation via Zod efficiently rejected malformed payloads before database interaction, saving database CPU cycles.

# Chapter VI: External API Test

---

## 6.1 REST Endpoint Validation

All endpoints were tested for proper JSON request/response formats and standard HTTP status code outputs.

| Method | Endpoint                              | Description                         | Test Result |
|--------|---------------------------------------|-------------------------------------|-------------|
| PATCH  | <code>/restaurants/{id}/verify</code> | Updates verification status.        | 200 OK      |
| GET    | <code>/moderation/media</code>        | Retrieves media pending moderation. | 200 OK      |
| DELETE | <code>/moderation/reviews/{id}</code> | Removes a review from the platform. | 200 OK      |
| POST   | <code>/reports</code>                 | Submits a new report.               | 201 Created |
| PATCH  | <code>/support/{id}</code>            | Updates a support ticket status.    | 200 OK      |

## Chapter VII: Security & Authorization Test

### 7.1 Authentication Validation

- **Mechanism:** JSON Web Tokens (JWT).
- **Test:** Requests made to `/moderation/*` without a valid JWT in the header.
- **Result:** System successfully returned `401 Unauthorized`.

### 7.2 Role-Based Access Control (RBAC) Test

- **Mechanism:** Restricting access based on User, Vendor, or Moderator roles.
- **Test:** A user with the "Vendor" role attempts to send a `DELETE` request to `/moderation/media/{id}`.
- **Result:** System successfully blocked the action, returning `403 Forbidden`.

## Chapter VIII: Conclusion

### 8.1 System Capacity

The layered architecture separates request handling, business logic, and data access cleanly. This design proved highly capable during testing, easily managing the CRUD operations for reports, media tickets, and support tickets.

## 8.2 System Limitations

Because the Moderation System references shared entities (Users, Restaurants, Reviews, Media) managed by other subsystems, network latency or downtime in those adjacent subsystems can briefly bottleneck moderation workflows.

# Chapter IX: Result Analysis

---

## 9.1 Features Confirmed

- **Automated Workflows:** The system correctly evaluates review thresholds and verifies restaurants autonomously without manual moderator intervention.
- **State Management:** Reports successfully transition through `Pending`, `Under Review`, `Resolved`, and `Rejected`. Support tickets accurately track `Pending`, `In Progress`, `Resolved`, and `Closed` states.

## 9.2 Identified Flaws & Future Enhancements

No critical bugs were discovered during testing. However, consistent with the SDD's future roadmap, implementing AI-assisted content moderation and automatic spam detection will further reduce manual moderator load.

# Chapter X: Resource Usage During the Test

---

## 10.1 Server Metrics

During the execution of the Stress Test (Chapter V) and External API Test (Chapter VI), the following resource usage was recorded:

- **Database Memory (PostgreSQL):** Peak usage remained under 60% of allocated buffer cache during heavy read/write cycles from the `Report` and `Support Ticket` entities.
- **Node.js / Express CPU:** Maintained stable event-loop execution. Validating payloads via Zod induced minimal CPU overhead (less than 15% utilization at peak load).
- **Storage (Media Buckets):** Private bucket transition tests handled simulated high-resolution image pointers without lagging the `MediaService` logic.